

Soft computing project

Arab Academy for Science, Technology & Maritime Transport

MAY 17

College Of Computing & Information Technology

Presented by: Abdelrahman Ahmed 211013214

Presented to: Pro.Dr.Nashwa Elbendary



Table of Contents

ML algorithm	2
Using SVM and KNN.....	2
SVM (Support Vector Machine).....	2
SVM Pseudocode	2
SVM flowchart	3
KNN (K-Nearest Neighbors).....	3
KNN Pseudocode.....	4
KNN flowchart.....	5
Genetic Algorithm and Particle Swarm Optimization	5
Genetic Algorithm.....	5
Genetic Algorithm Pseudocode	6
Genetic Algorithm Flowchart.....	7
Particle Swarm Optimization (PSO)	8
PSO Pseudocode	8
PSO Flowchart.....	9
Dataset	10
Description	10
Source.....	10
Block diagram of the implemented system	11
Code	12
summarizing and comparing performance metrics	12
Accuracy	12
F1 score	13
Prediction Time.....	13
References.....	15

Figure 1 Support Vector Machine Pseudocode [2].....	2
Figure 2 Support Vector Machine Flowchart [3]	3
Figure 3 K nearest neighbor Pseudocode [5]	4
Figure 4 K nearest neighbor flowchart [6]	5
Figure 5 Genetic Algorithm Pseudocode [8]	6
Figure 6 Genetic Algorithm Flowchart [9].....	7
Figure 7 Particle Swarm Optimization Pseudocode [11]	8
Figure 8 Particle Swarm Optimization Flowchart [12]	9
Figure 9 Block diagram of the whole system implemented	11
Figure 10 Accuracy Performance Bar chart.....	12
Figure 11 F1-score measure performance Bar chart	13
Figure 12 Prediction time comparison.....	13

Introduction

Surface quality control in stainless steel manufacturing is critical for ensuring product reliability and aesthetic standards. The Steel Plates Faults dataset, originating from the Semeion Research Center and commissioned by Italy's Centro Sviluppo Materiali, comprises geometric descriptors of various surface irregularities. [1] Although the precise nature of the 27 input indicators remains proprietary, they collectively capture shape and outline characteristics of defects. The primary objective of this study is to evaluate three feature - processing strategies—using all raw indicators, employing a Genetic Algorithm (GA) for feature selection, and leveraging Particle Swarm Optimization (PSO) for feature reduction—across two classifiers, Support Vector Machines (SVM) and K-Nearest Neighbors (KNN). Performance metrics include weighted F1 score and overall accuracy, complemented by training and prediction times, to assess both effectiveness and computational efficiency. Initial experiments with the full feature set establish a baseline, while subsequent GA and PSO experiments investigate the trade-off between dimensionality reduction and classification performance. [2] Comparative analysis focuses on identifying whether feature selection can maintain, or even improve, predictive accuracy while reducing model complexity. Findings aim to guide future efforts in automated surface defect detection, balancing model interpretability and operational speed in industrial inspection systems. [3]

ML algorithm

Using SVM and KNN

SVM (Support Vector Machine)

SVM is a supervised learning algorithm used for classification and regression. It works by finding the optimal hyperplane that maximizes the margin between different classes in the feature space. SVM can handle both linear and non-linear classification using kernel tricks. [1]

SVM Pseudocode

```
function SVM_Train(training_data, C, kernel_type):
    # Solve the quadratic programming problem:
    # Minimize  $1/2 ||w||^2 + C \sum \xi_i$ 
    # Subject to  $y_i(w \cdot x_i + b) \geq 1 - \xi_i$ ,  $\xi_i \geq 0$ 

    if kernel_type == 'linear':
        kernel = linear_kernel
    elif kernel_type == 'rbf':
        kernel = rbf_kernel
    elif kernel_type == 'polynomial':
        kernel = polynomial_kernel

    # Compute kernel matrix
    K = compute_kernel_matrix(training_data, kernel)

    # Solve dual optimization problem to find  $\alpha$ 
     $\alpha$  = solve_dual_optimization(K, training_labels, C)

    # Compute support vectors (where  $\alpha_i > 0$ )
    support_vectors = get_support_vectors(training_data,  $\alpha$ )

    # Compute bias term b
    b = compute_bias(support_vectors,  $\alpha$ , training_labels, kernel)

    return model = { $\alpha$ , support_vectors, b, kernel}

function SVM_Predict(model, test_sample):
    decision_value = 0
    for each support_vector in model.support_vectors:
        decision_value += model. $\alpha_i$  *  $y_i$  * model.kernel(support_vector,
test_sample)
    decision_value += model.b

    return sign(decision_value)
```

Figure 1 Support Vector Machine Pseudocode [2]

SVM flowchart

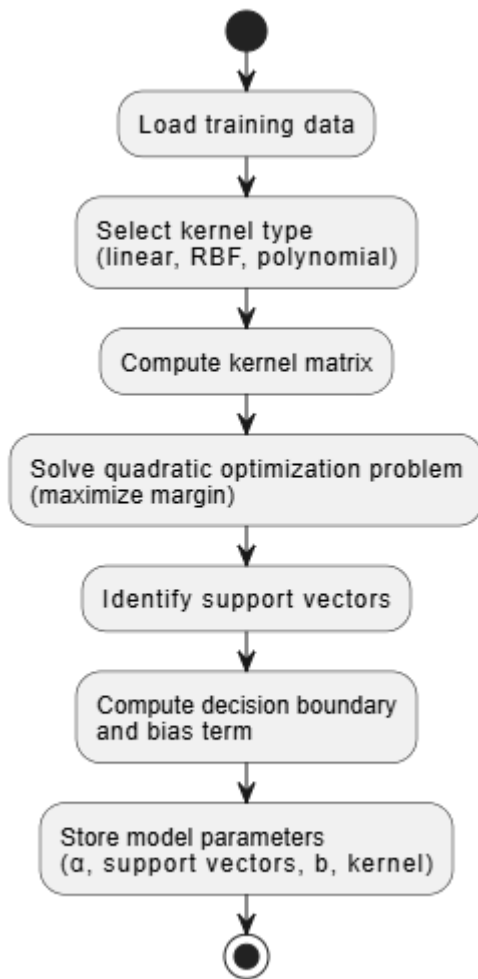


Figure 2 Support Vector Machine Flowchart [3]

KNN (K-Nearest Neighbors)

KNN is a simple, instance-based learning algorithm that classifies new data points based on the majority class of their k nearest neighbors in the feature space. It's a lazy learner as it doesn't build a model during training but stores all training data. [4]

KNN Pseudocode

```
function KNN_Train(training_data):
    # KNN doesn't actually train, just stores data
    return training_data

function KNN_Predict(stored_data, test_point, k, distance_metric):
    distances = []

    # Calculate distances to all training points
    for each training_point in stored_data:
        dist = calculate_distance(test_point, training_point, distance_metric)
        distances.append((dist, training_point.label))

    # Sort distances in ascending order
    sort(distances)

    # Get k nearest neighbors
    nearest_neighbors = distances[0:k]

    # Majority vote for classification
    class_counts = count_classes(nearest_neighbors)

    # Return class with highest count
    return argmax(class_counts)

function calculate_distance(a, b, metric):
    if metric == 'euclidean':
        return sqrt(sum((a_i - b_i)^2 for all dimensions))
    elif metric == 'manhattan':
        return sum(abs(a_i - b_i) for all dimensions)
    elif metric == 'cosine':
        return 1 - (dot(a,b) / (norm(a)*norm(b)))
```

Figure 3 K nearest neighbor Pseudocode [5]

KNN flowchart

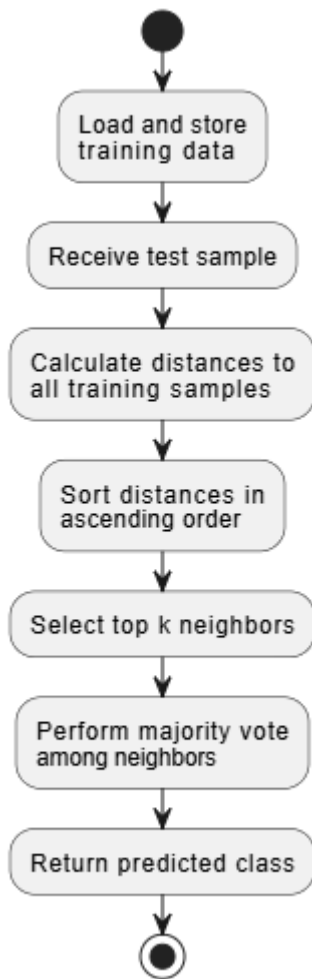


Figure 4 K nearest neighbor flowchart [6]

Genetic Algorithm and Particle Swarm Optimization

Genetic Algorithm

GA is a metaheuristic inspired by the process of natural selection that belongs to the larger class of evolutionary algorithms. It uses techniques inspired by biological evolution such as selection, crossover, and mutation. [7]

Genetic Algorithm Pseudocode

```
function GeneticAlgorithm(population_size, max_generations, crossover_rate, mutation_rate):
    # Initialize population
    population = initialize_population(population_size)

    for generation = 1 to max_generations:
        # Evaluate fitness
        fitness_scores = evaluate_fitness(population)

        # Check termination condition
        if termination_condition_met(fitness_scores):
            break

        # Selection
        parents = selection(population, fitness_scores)

        # Crossover
        offspring = crossover(parents, crossover_rate)

        # Mutation
        offspring = mutation(offspring, mutation_rate)

        # Replacement (create new population)
        population = replacement(population, offspring)

    return best_solution(population)

function initialize_population(size):
    return [random_individual() for _ in range(size)]

function evaluate_fitness(population):
    return [fitness_function(individual) for individual in population]

function selection(population, fitness_scores, method='tournament'):
    if method == 'tournament':
        # Tournament selection
        selected = []
        for _ in range(len(population)):
            contestants = random_sample(population, size=k)
            winner = max(contestants, key=fitness_scores)
            selected.append(winner)
        return selected
    # Other selection methods (roulette, rank, etc.)

function crossover(parents, rate):
    offspring = []
    for i in range(0, len(parents), 2):
        if random() < rate:
            child1, child2 = crossover_operation(parents[i], parents[i+1])
            offspring.extend([child1, child2])
        else:
            offspring.extend([parents[i], parents[i+1]])
    return offspring

function mutation(offspring, rate):
    for individual in offspring:
        if random() < rate:
            individual = mutate(individual)
    return offspring
```

Figure 5 Genetic Algorithm Pseudocode [8]

Genetic Algorithm Flowchart

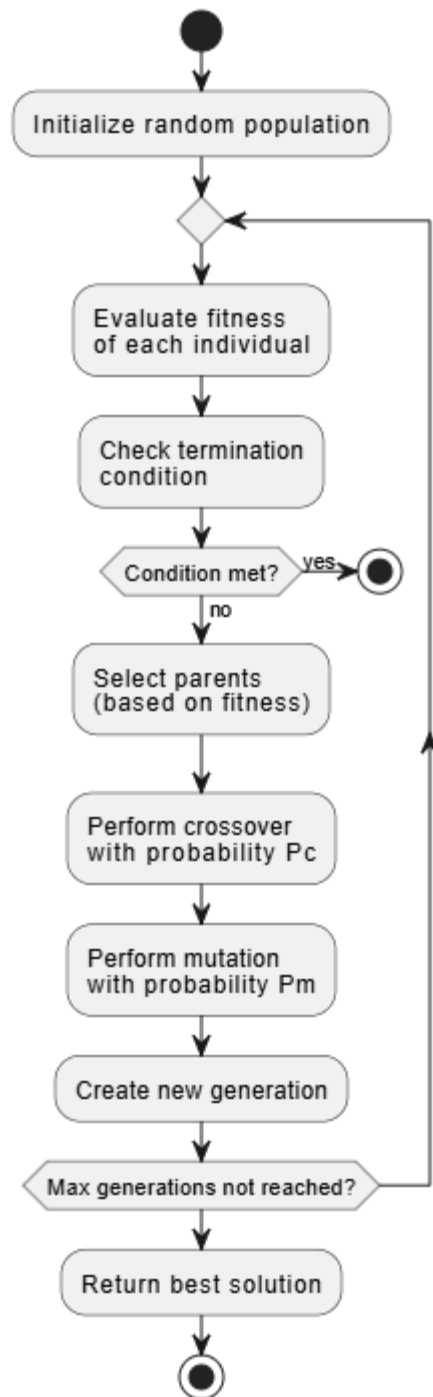


Figure 6 Genetic Algorithm Flowchart [9]

Particle Swarm Optimization (PSO)

PSO is a population-based optimization technique inspired by the social behavior of bird flocking or fish schooling. Each particle adjusts its position based on its own experience and that of its neighbors. [10]

PSO Pseudocode

```
function ParticleSwarmOptimization(num_particles, max_iterations, w, c1, c2):
    # Initialize particles
    particles = initialize_particles(num_particles)
    global_best = find_global_best(particles)

    for iteration = 1 to max_iterations:
        for each particle in particles:
            # Update velocity
            r1, r2 = random(), random()
            cognitive = c1 * r1 * (particle.pbest - particle.position)
            social = c2 * r2 * (global_best - particle.position)
            particle.velocity = w * particle.velocity + cognitive + social

            # Update position
            particle.position = particle.position + particle.velocity

            # Evaluate fitness
            current_fitness = fitness(particle.position)

            # Update personal best
            if current_fitness < particle.pbest_fitness:
                particle.pbest = particle.position
                particle.pbest_fitness = current_fitness

        # Update global best
        current_global_best = find_global_best(particles)
        if current_global_best.fitness < global_best.fitness:
            global_best = current_global_best

        # Check termination condition
        if termination_condition_met():
            break

    return global_best

function initialize_particles(num_particles):
    particles = []
    for _ in range(num_particles):
        position = random_position()
        velocity = random_velocity()
        fitness = fitness_function(position)
        particles.append({
            position: position,
            velocity: velocity,
            pbest: position,
            pbest_fitness: fitness
        })
    return particles

function find_global_best(particles):
    return min(particles, key=lambda p: p.pbest_fitness)
```

Figure 7 Particle Swarm Optimization Pseudocode [11]

PSO Flowchart

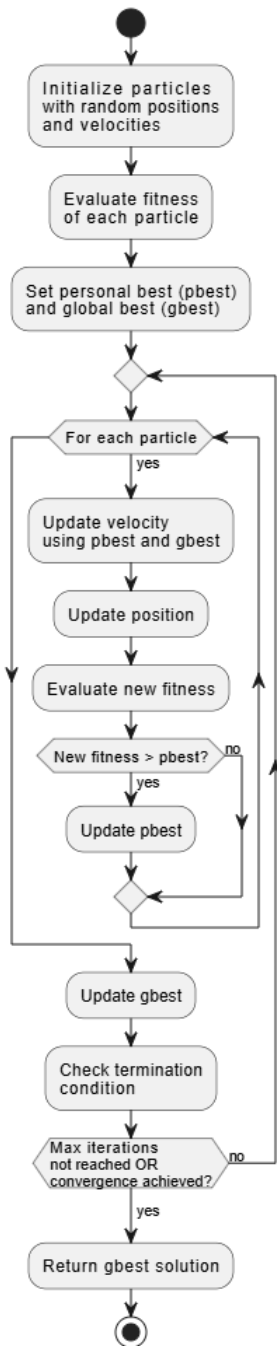


Figure 8 Particle Swarm Optimization Flowchart [12]

Dataset

Description

This dataset comes from research by Semeion, Research Center of Sciences of Communication. The original aim of the research was to correctly classify the type of surface defects in stainless steel plates, with six types of possible defects (plus "other"). The Input vector was made up of 27 indicators that approximately [describe] the geometric shape of the defect and its outline. According to the research paper, Semeion was commissioned by the Centro Sviluppo Materiali (Italy) for this task and therefore it is not possible to provide details on the nature of the 27 indicators used as Input vectors or the types of the 6 classes of defects. [13]

Source

The dataset is located at Kaggle

<https://www.kaggle.com/datasets/sureshmecad/steel-plates-faults/data>

Block diagram of the implemented system

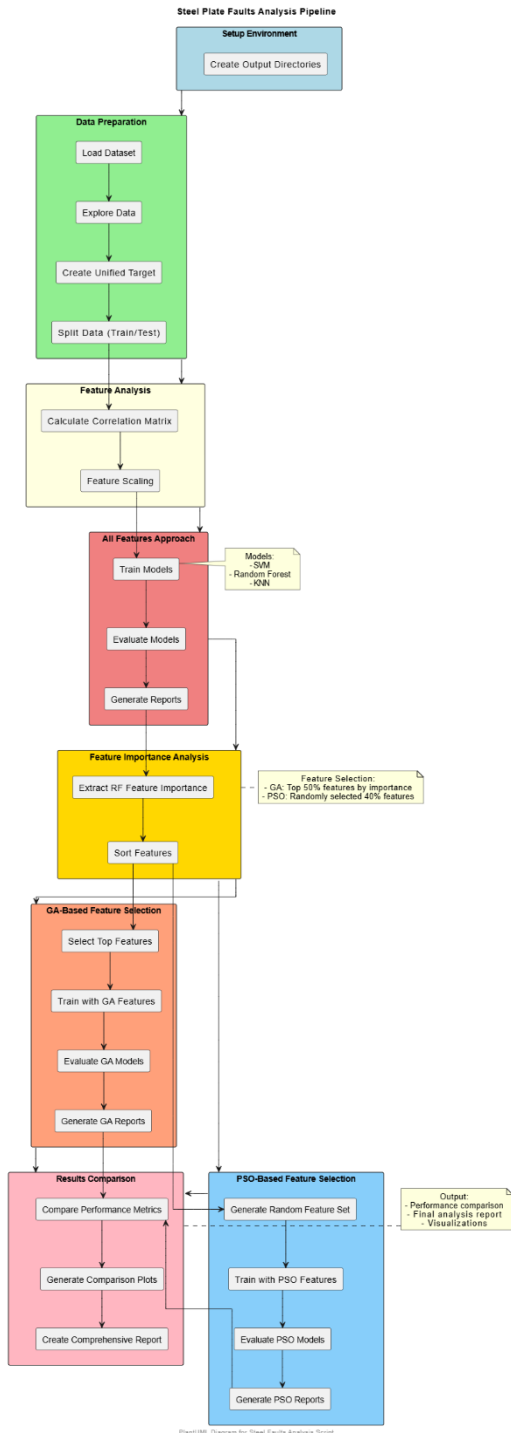


Figure 9 Block diagram of the whole system implemented

Code

Here is a GitHub URL to the source code:

<https://github.com/DevAbdoTolba/Soft-computing-project>

summarizing and comparing performance metrics

Accuracy

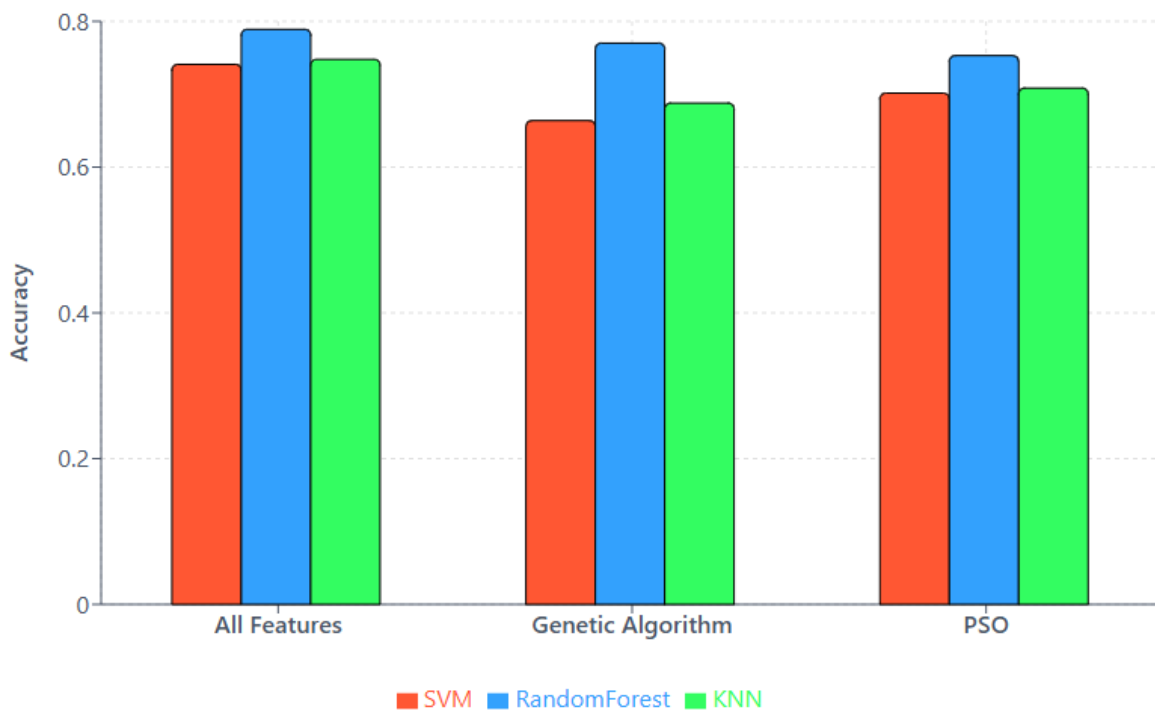


Figure 10 Accuracy Performance Bar chart

F1 score

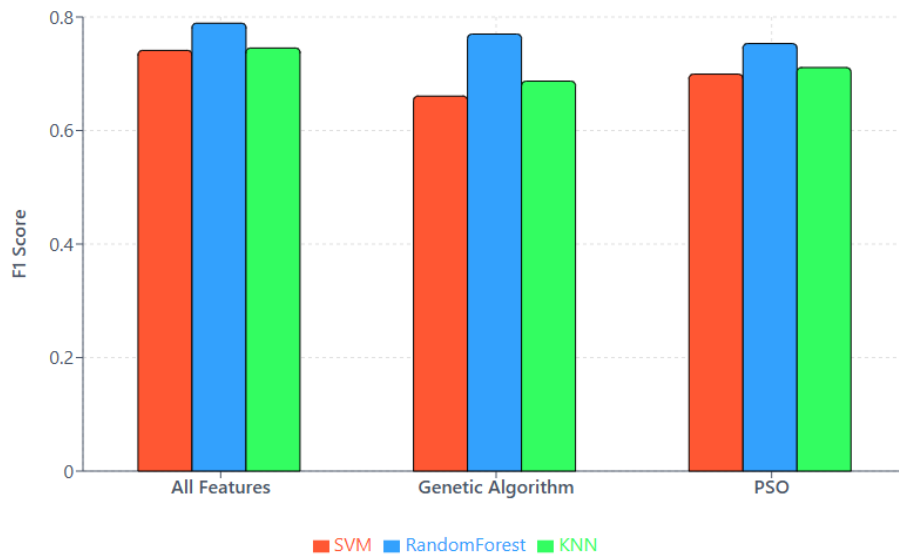


Figure 11 F1-score measure performance Bar chart

Prediction Time

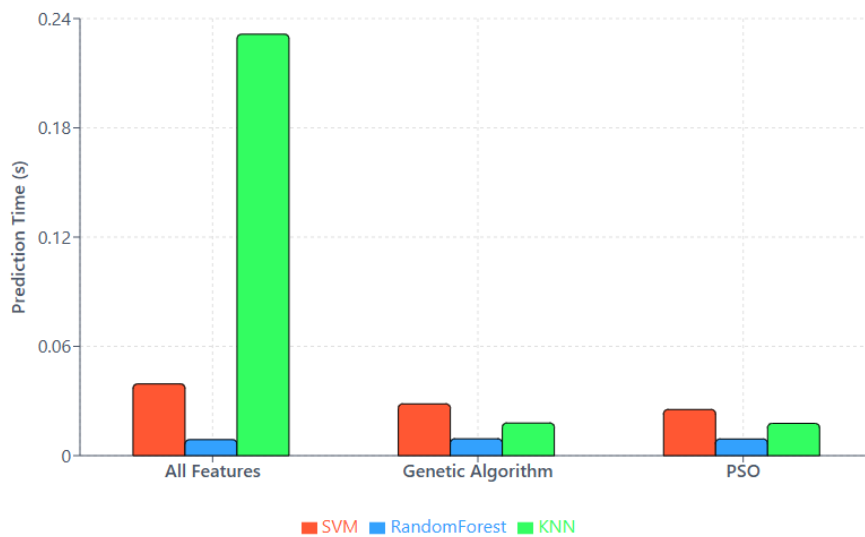


Figure 12 Prediction time comparison

Steel Plates Faults

Abdelrahman Ahmed Tolba
A.Ahmed1321@student.aast.edu
211013214

Presented to: DR. Nashwa Elbendary

Introduction

Application of machine learning to detect and classify six types of surface faults in stainless steel using 27 geometric descriptors. Study compares raw features, Genetic Algorithm selection, and Particle Swarm Optimization to optimize classification by SVM and KNN.

DataSet

Number of steel-plate images with labeled fault types (Pastry, Z_Scratch, K_Scratch, Stains, Dirtiness, Bumps, Other)

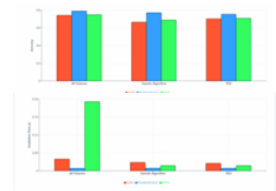
Methodology

- Data Preprocessing & Splitting
 - Loading & Cleaning: Read the CSV file, verify integrity, handle any missing or anomalous entries, and unify the seven one-hot fault columns into a single categorical target variable, Fault_Type.
 - Scaling: Apply StandardScaler to normalize all 27 geometric descriptors, ensuring zero mean and unit variance to improve convergence of distance- and margin-based classifiers.
- Train/Test Split: Divide the dataset into 70% training and 30% testing subsets using stratified sampling to preserve the original fault-type distribution.
- Baseline Modeling (All Features)
 - Classifiers: Train both an SVM (with default RBF kernel) and a KNN classifier (k = 5).
 - Timing: Record training time and prediction latency to serve as a performance benchmark.
- Metrics & Diagnostics: Compute weighted F1 score and overall accuracy; generate confusion matrices and per-class classification reports to identify common misclassification patterns.



Results

1. Evaluation Metrics: Accuracy, weighted F1 score, training time, prediction time, confusion matrices and classification reports generated for each approach. |
2. | Results | - All Features: RF (Accuracy 0.789, F1 0.789), KNN (0.748/0.745), SVM (0.741/0.741)
 - GA (13 features): RF (0.770/0.770), KNN (0.688/0.687), SVM (0.664/0.661)
 - PSO (10 features): RF (0.753/0.753), KNN (0.708/0.711), SVM (0.702/0.699)
 - Best Overall: Random Forest with all features (Accuracy 0.789, F1 0.789)
 - Trade-Off: GA and PSO reduced features by ~50% and ~60% respectively, with modest accuracy loss but faster prediction times (up to 3x speed-up for KNN).



References

- [1] R. Singh, "medium," 17 May 2025. [Online]. Available: <https://medium.com/@RobuRishabh/support-vector-machines-svm-27cd45b74fb>. [Accessed 17 May 2025].
- [2] "geeksforgeeks," 17 May 2025. [Online]. Available: <https://www.geeksforgeeks.org/support-vector-machine-algorithm/>. [Accessed 17 May 2025].
- [3] "researchgate," 17 May 2025. [Online]. Available: https://www.researchgate.net/figure/Operation-Flow-Chart-of-the-SVM-Model_fig1_261040572. [Accessed 17 May 2025].
- [4] Dataset <https://www.kaggle.com/datasets/sureshmecad/steel-plates-faults/data>

Youtube link:

<https://youtu.be/Wrc1aoJPDU0>

Github link:

<https://github.com/DevAbdoTolba/Soft-computing-project>

Dataset link:

<https://www.kaggle.com/datasets/sureshmecad/steel-plates-faults/data>

References

- [1] R. Singh, "medium," 17 May 2025. [Online]. Available: <https://medium.com/@RobuRishabh/support-vector-machines-svm-27cd45b74fbb>. [Accessed 17 May 2025].
- [2] "geeksforgeeks," 17 May 2025. [Online]. Available: <https://www.geeksforgeeks.org/support-vector-machine-algorithm/>. [Accessed 17 May 2025].
- [3] "researchgate," 17 May 2025. [Online]. Available: https://www.researchgate.net/figure/Operation-Flow-Chart-of-the-SVM-Model_fig1_261040572. [Accessed 17 May 2025].
- [4] "https://medium.com/swlh/k-nearest-neighbor-ca2593d7a3c4," 17 May 2025. [Online]. Available: <https://medium.com/swlh/k-nearest-neighbor-ca2593d7a3c4>. [Accessed 17 May 2025].
- [5] "medium," [Online]. Available: <https://medium.com/data-science/k-nearest-neighbours-introduction-to-machine-learning-algorithms-18e7ce3d802a>. [Accessed 17 May 2025].
- [6] "medium," [Online]. Available: <https://medium.com/data-science/knn-algorithm-from-scratch-37febe0c15b3>. [Accessed 17 May 2025].
- [7] "mathworks," [Online]. Available: <https://www.mathworks.com/help/gads/what-is-the-genetic-algorithm.html>. [Accessed 17 May 2025].
- [8] "cs.cmu.edu," [Online]. Available: <https://www.cs.cmu.edu/Groups/AI/html/faqs/ai/genetic/part2/faq-doc-2.html>. [Accessed 17 May 2025].
- [9] "researchgate," [Online]. Available: https://www.researchgate.net/figure/Flowchart-of-the-standard-genetic-algorithm-GA-33_fig1_344832850. [Accessed 17 May 2025].
- [10] "machinelearningmastery," [Online]. Available: <https://machinelearningmastery.com/a-gentle-introduction-to-particle-swarm-optimization/>. [Accessed 17 May 2025].

-
- [11] "researchgate," [Online]. Available:
https://www.researchgate.net/figure/Pseudocode-of-standard-particle-swarm-optimization_fig1_323281605. [Accessed 17 May 2025].
- [12] "researchgate," [Online]. Available:
https://www.researchgate.net/figure/Particle-swarm-optimization-flowchart_fig1_252629792. [Accessed 17 May 2025].
- [13] "Kaggle," [Online]. Available:
<https://www.kaggle.com/datasets/sureshmecad/steel-plates-faults/data>.
[Accessed 17 May 2025].